

Análisis de Rendimiento: Suma Paralela de Vectores con OpenMP

Este documento presenta un desglose detallado de la implementación en C utilizando la API OpenMP para paralelizar la suma de dos vectores. Se incluye el código fuente debidamente comentado, una explicación paso a paso de los componentes críticos y la interpretación analítica de los resultados de ejecución.

1. Código Fuente Comentado

A continuación se presenta el código optimizado y comentado, detallando el propósito de cada sección técnica:

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h> // Librería esencial para utilizar las funciones de OpenMP

void medir_rendimiento(int N, int max_hilos) {
    // 1. ASIGNACIÓN DINÁMICA DE MEMORIA
    // Usamos malloc porque los vectores masivos exceden el límite de la pila (Stack).
    // Esto previene errores de "Stack Overflow".
    int *A = (int *)malloc(N * sizeof(int));
    int *B = (int *)malloc(N * sizeof(int));
    int *C = (int *)malloc(N * sizeof(int));

    if (A == NULL || B == NULL || C == NULL) {
        printf("Error asignando memoria para N = %d\n", N);
        return;
    }

    // 2. INICIALIZACIÓN DE LOS VECTORES
    // Inicialización de arreglos utilizando índices basados en 0
    #pragma omp parallel for
    for (int i = 0; i < N; i++) {
        A[i] = i;
        B[i] = N - i;
    }

    printf("\n=====n");
    printf("Prueba con N = %d componentes\n", N);
    printf("=====n");

    // 3. EJECUCIÓN SECUENCIAL (Línea base)
    double inicio_sec = omp_get_wtime(); // Marca de tiempo en reloj de pared (wall-clock)
```

```

for (int i = 0; i < N; i++) {
    C[i] = A[i] + B[i];
}

double tiempo_sec = omp_get_wtime() - inicio_sec;
printf("Secuencial: \t\t%f segundos\n", tiempo_sec);

// 4. EJECUCIÓN PARALELA Y EVALUACIÓN DE HILOS
int num_hilos_a_probar[] = {2, 4, 8, 16};
int cantidad_pruebas = sizeof(num_hilos_a_probar) / sizeof(num_hilos_a_probar[0]);

for (int t = 0; t < cantidad_pruebas; t++) {
    int hilos = num_hilos_a_probar[t];

    // Evitamos ejecutar más hilos lógicos de los que el hardware soporta
    if (hilos > max_hilos) break;

    omp_set_num_threads(hilos); // Asignación dinámica de hilos

    double inicio_par = omp_get_wtime();

    // 5. DIRECTIVA DE PARALELIZACIÓN (Fork-Join Model)
    // Divide el espacio de iteración (0 a N-1) en chunks asignados a cada hilo.
    #pragma omp parallel for
    for (int i = 0; i < N; i++) {
        C[i] = A[i] + B[i];
    }

    double tiempo_par = omp_get_wtime() - inicio_par;

    // 6. CÁLCULO DE ACELERACIÓN (Speedup)
    double speedup = tiempo_sec / tiempo_par;

    printf("Paralelo ( %2d hilos): \t%f segundos \t(Aceleración: %.2fx)\n",
        hilos, tiempo_par, speedup);
}

// 7. LIBERACIÓN DE MEMORIA
// Previene memory leaks en la ejecución
free(A);
free(B);
free(C);
}

int main() {
    int max_hilos = omp_get_max_threads();
    printf("Hilos maximos soportados por tu CPU: %d\n", max_hilos);

    // Tamaños escalonados en potencias de 10
    int tamanos_N[] = {1000000, 10000000, 100000000};
    int num_tamanos = sizeof(tamanos_N) / sizeof(tamanos_N[0]);

```

```
for(int i = 0; i < num_tamamos; i++) {  
    medir_rendimiento(tamamos_N[i], max_hilos);  
}  
  
return 0;  
}
```

2. Explicación Paso a Paso

Gestión de Memoria y Entorno

El código inicializa el entorno importando `<omp.h>`, una cabecera que expone rutinas clave de la API de OpenMP. Destaca el uso de `omp_get_wtime()`, el cual ofrece mediciones temporales de alta resolución independientes del tiempo de CPU consumido, y `omp_set_num_threads()`, que regula explícitamente el grado de concurrencia.

La estructura de los datos exige el uso de asignación dinámica (`malloc`). Reservar arreglos en memoria dinámica previene desbordamientos de pila; intentar instanciar arreglos estáticos de más de algunos megabytes en C resultaría en un fallo de segmentación (*Segmentation Fault*) debido a la limitación de la pila de llamadas (Stack).

Modelo de Paralelización

El pragmatismo de OpenMP radica en directivas como `#pragma omp parallel for`. Al encontrar esta instrucción, el programa entra en el modelo de ejecución *Fork-Join*: el hilo maestro genera (fork) un grupo de hilos esclavos. El bucle se divide equitativamente; si `N` es 10 millones y se usan 2 hilos, el primer hilo procesa los índices del 0 al 4.999.999 y el segundo de 5.000.000 al 9.999.999. Al terminar el bucle, todos se sincronizan en una barrera implícita (join) antes de continuar el flujo.

Evaluación de Aceleración

La métrica *Speedup* evalúa el incremento de rendimiento relativo, determinando cuántas veces más rápida resulta la operación concurrente comparada con el cálculo estrictamente algorítmico y lineal.

3. Análisis e Interpretación de Resultados

A partir de la ejecución capturada en terminal, el procesador confirma disponer de un tope lógico de **4 hilos**. El comportamiento del sistema por volumen de carga ilustra conceptos clave del procesamiento de alto rendimiento:

Carga Ligera (N = 1.000.000 componentes)

- **Secuencial:** 0.002387 s
- **Paralelo 4 hilos:** 0.003942 s (Aceleración 0.61x)

Interpretación: Observamos una degradación del rendimiento con 4 hilos. La carga computacional es demasiado breve. El *Overhead* (tiempo invertido por el planificador del SO en crear, asignar y destruir hilos) supera con creces la ventaja matemática. La anomalía estadística a 2 hilos (4.20x) es típica de la arquitectura de la caché: es posible que la porción de memoria evaluada se haya alineado perfectamente con los bloques de caché L2/L3 o el procesador experimentara un pico de Turbo Boost asimétrico.

Carga Media (N = 10.000.000 componentes)

- **Secuencial:** 0.013035 s
- **Paralelo 2 hilos:** 0.007145 s (Aceleración 1.82x)
- **Paralelo 4 hilos:** 0.006200 s (Aceleración 2.10x)

Interpretación: La operación alcanza un punto operativo ideal. El overhead es ahora insignificante respecto al tiempo de cómputo. Con 2 hilos la eficiencia es alta, con un escalado casi lineal (1.82x). Al activar 4 hilos (2.10x) la aceleración continúa creciendo pero la eficiencia por hilo decrece, demostrando que empezamos a someter los buses de la placa base a estrés.

Carga Masiva (N = 100.000.000 componentes)

- **Secuencial:** 0.118819 s
- **Paralelo 2 hilos:** 0.049375 s (Aceleración 2.41x)

Interpretación: A esta escala el paralelismo se consolida, pero el crecimiento sub-lineal revela un nuevo cuello de botella: el límite del *Ancho de banda de Memoria*. Aunque la CPU puede sumar números a velocidades extremas, la memoria RAM tiene una tasa máxima de transferencia para proporcionar los valores de `A[i]` y `B[i]`. Inyectar más hilos no resolvería la latencia impuesta por el bus de la memoria.